

Marco teórico del control de acceso en SIPLAP Booked

1 Fundamentos del Sistema de Planeación Institucional

El Sistema de Planeación Institucional de la Universidad de Oriente requiere una base de identidades y autorizaciones que permita relacionar las actividades de planeación con las responsabilidades de su personal. El anteproyecto sitúa esta necesidad en los módulos de usuarios, perfiles, departamentos, roles, permisos, permisos por rol, estatus y tipos de usuario. Su propósito comprende analizar, diseñar, programar, implementar y documentar estos componentes para sostener la seguridad y el flujo operativo de la plataforma (Euan Be, s. f., pp. 2-6). Este marco teórico explica los conceptos que justifican su organización y su aplicación al proyecto siplap-booked.

La planeación, entendida aquí como la relación entre objetivos, actividades y recursos institucionales, necesita información atribuible a personas identificadas. Una actividad puede vincularse con un objetivo estratégico, mientras que su ejecución exige responsables y recursos disponibles. En el esquema de datos del proyecto aparecen ejes, objetivos, actividades, espacios, recursos y reservaciones. Estas entidades proporcionan el contexto funcional del control de acceso: las operaciones sobre ellas deben corresponder con responsabilidades institucionales, aunque sus servicios completos no forman parte de todos los módulos desarrollados en la versión examinada.

Desde esta perspectiva, el sistema puede analizarse como un conjunto de componentes interdependientes. La cuenta identifica a quien actúa; el departamento describe su adscripción; el rol representa una función; el permiso delimita una operación. Si estas categorías se confunden, la aplicación puede atribuir facultades a partir de información que únicamente describe al personal. Un cargo registrado en el perfil, por ejemplo, necesita una asignación de autorización verificable antes de producir acceso administrativo.

El fundamento del proyecto es, por tanto, una correspondencia explícita entre identidad, estructura organizacional y capacidad de actuación. La tecnología materializa esa correspondencia mediante relaciones de datos y comprobaciones del servidor. Su utilidad debe evaluarse por la posibilidad de explicar quién puede ejecutar una acción y bajo qué condiciones, además de por la existencia de pantallas de administración.

La exposición parte del ámbito institucional, desarrolla los modelos de seguridad y explica las decisiones de arquitectura, persistencia, interfaz y verificación. Los ejemplos de políticas representan aplicaciones razonadas al proyecto; su adopción requiere validación institucional. Esta distinción permite utilizar la teoría como fundamento del diseño sin convertir escenarios ilustrativos en disposiciones oficiales de la Universidad.

2 Sistemas de información y coordinación organizacional

Un sistema de información puede describirse operativamente como la articulación de personas, procedimientos, datos y medios técnicos para producir información útil. En el proyecto, esa articulación se observa cuando una solicitud administrativa se transforma en un registro estructurado, se asocia con un responsable y se procesa mediante reglas. La aportación del software consiste en hacer verificables las relaciones que sostienen el procedimiento. Digitalizar un formulario sin definir quién puede consultarlo o modificarlo conserva una parte importante de la ambigüedad organizacional.

Para analizar SIPLAP Booked conviene diferenciar datos, información y decisión. El identificador de una cuenta es un dato; la lista de cuentas activas de un departamento constituye información organizada; autorizar a una persona para administrar registros representa una decisión institucional. La base de datos almacena las decisiones mediante asignaciones de roles y permisos, pero no determina por sí sola su legitimidad. La autoridad responsable debe establecer el criterio que el programa ejecutará.

La coordinación organizacional aparece en las dependencias entre módulos. Un rol inexistente no puede asignarse válidamente a un usuario. Una persona sin cuenta no puede identificarse como autora de una operación. Un departamento puede servir para delimitar el universo de registros que una función tiene permitido gestionar. Estas dependencias justifican un diseño integral, en lugar de considerar cada catálogo como una colección aislada de altas y bajas.

El anteproyecto plantea que los módulos de acceso sostendrán posteriores flujos de validación, logística y reportes (Euan Be, s. f., p. 6). En términos de diseño, esto significa que una decisión local puede producir consecuencias en otras áreas. Desactivar una cuenta debe impedir futuras actuaciones de esa identidad, mientras que borrar su registro puede afectar referencias históricas. La continuidad de la información exige distinguir el retiro de una autorización de la eliminación de sus antecedentes.

También existe una relación entre centralización y responsabilidad. Un catálogo común evita que cada módulo invente identidades o nombres de roles diferentes. Sin embargo, concentrar la información exige controles claros sobre quién administra ese catálogo. La consistencia organizacional depende de que el sistema mantenga una definición común de sus entidades y de que los cambios relevantes puedan explicarse.

Aplicado al proyecto, este enfoque permite interpretar el control de acceso como un componente transversal. Sus reglas acompañan al usuario durante las operaciones y conectan la estructura administrativa con el comportamiento del servidor. Su función es mantener una correspondencia estable entre responsabilidades reales y acciones digitales, incluso cuando cambian las personas que ocupan un puesto.

3 Seguridad de la información y activos del proyecto

La seguridad de la información se estudia a partir de las propiedades que deben conservar los activos de un sistema. Para SIPLAP Booked, los activos incluyen credenciales, datos personales, asignaciones de roles, permisos y registros institucionales. La confidencialidad exige limitar su conocimiento a sujetos autorizados; la integridad requiere impedir modificaciones indebidas; la disponibilidad supone que las funciones necesarias puedan utilizarse cuando corresponde. Estas propiedades ofrecen criterios para analizar el efecto de una falla, más allá de su apariencia en la interfaz.

La confidencialidad se vulneraría si una respuesta de consulta incluyera el hash de una contraseña o datos personales innecesarios. La integridad se comprometería si una cuenta operativa pudiera asignarse un rol administrativo. La disponibilidad se vería afectada si se eliminaran todas las cuentas capaces de administrar el sistema. Estos escenarios muestran que el control de acceso influye en las tres propiedades y que las medidas deben atender tanto las lecturas como las modificaciones.

Los principios de protección de Saltzer y Schroeder (1975) incluyen mínimo privilegio, mediación completa y decisiones de acceso restrictivas por defecto. Su aportación conceptual consiste en diseñar la seguridad como parte de la estructura del sistema. Para el proyecto, esos principios orientan tres preguntas: qué facultades necesita una cuenta, dónde se comprueba cada solicitud y cómo se comporta la aplicación cuando una regla está ausente o no puede evaluarse.

El análisis de riesgos relaciona un activo con una amenaza, una condición vulnerable y sus consecuencias. Por ejemplo, una asignación administrativa es el activo; una petición manipulada es el medio de ataque; la ausencia de comprobación en el servidor es la vulnerabilidad; la modificación de roles es una consecuencia posible. Esta cadena permite justificar controles concretos sin presentar cualquier error como un incidente confirmado.

La superficie de exposición incluye los puntos por los que llegan datos al sistema. Los formularios, las rutas HTTP y la conexión entre cliente y servidor forman parte de ella. Una validación visual puede ayudar al usuario, pero el servidor necesita comprobar la solicitud recibida porque un cliente alternativo puede omitir el formulario. La persistencia agrega otra frontera: las restricciones de la base de datos deben impedir estados incoherentes incluso ante operaciones simultáneas.

La aplicación de estos conceptos exige reconocer límites. La presencia de autenticación y restricciones administrativas permite describir controles específicos, pero no demuestra seguridad absoluta ni una certificación. La teoría aporta criterios para diseñar y revisar el proyecto; la evidencia de funcionamiento requiere pruebas observables y evaluación de las condiciones reales de operación.

4 Identificación autenticación y autorización

La identificación consiste en declarar una identidad; la autenticación comprueba la evidencia asociada con ella; la autorización determina las operaciones permitidas. OWASP (s. f.-a) distingue autenticación y autorización porque una identidad válida no implica acceso a todos los recursos. En una aplicación institucional, estas etapas forman una secuencia conceptual, aunque sus comprobaciones puedan ejecutarse en diferentes componentes. Registrar un correo identifica una cuenta, comparar una contraseña permite autenticarla y evaluar sus roles permite resolver una solicitud de acceso.

En el proyecto, LoginUseCase normaliza el correo recibido, consulta la cuenta, compara la contraseña y verifica que el estatus sea ACTIVE. Después solicita la emisión de un token. Esta secuencia muestra una separación entre demostrar conocimiento de la credencial y conservar la elegibilidad de la cuenta. Una contraseña correcta no debe habilitar el ingreso de una cuenta desactivada; la condición administrativa representa una restricción adicional sobre la identidad.

La autorización se vuelve observable cuando una cuenta autenticada intenta una operación protegida. Los controladores administrativos examinados aplican JwtAuthGuard y SuperUserGuard. El primero verifica la evidencia de autenticación; el segundo exige el rol SUPERUSUARIO. De este modo, el ingreso de una cuenta activa y el acceso a la administración tienen condiciones diferentes. La distinción resulta necesaria para que el sistema pueda atender usuarios operativos sin concederles capacidad de administrar identidades.

Puede ilustrarse esta separación con tres peticiones sobre usuarios. Una petición sin token carece de evidencia de autenticación. Una petición con token válido de una cuenta operativa identifica a un sujeto, pero no satisface la regla administrativa. Una petición de un superusuario activo puede superar ambos controles y continuar hacia la validación del contenido. Cada resultado corresponde a una causa diferente y debe comunicarse de manera consistente.

La autorización tampoco se agota al comprobar el tipo de operación. Si un futuro servicio permite consultar actividades del propio departamento, deberá evaluar la relación entre el solicitante y el registro. Tener capacidad de lectura de actividades no equivale necesariamente a consultar todas las actividades institucionales. Este análisis distingue autorización de función y autorización sobre objetos concretos.

Para fundamentar el diseño, conviene describir cada operación con su sujeto, recurso, acción y condición. Esta formulación evita reglas implícitas basadas en la pantalla desde la que se envió la petición. En SIPLAP Booked permite explicar de manera verificable por qué el servidor acepta o rechaza una acción y qué información institucional participa en esa decisión.

5 Modelos de control de acceso y pertinencia de RBAC

Los modelos de control de acceso expresan criterios distintos para decidir quién puede actuar sobre un recurso. El control discrecional relaciona permisos con decisiones del propietario o de quien administra el recurso. El control obligatorio utiliza políticas centrales y categorías de seguridad. El control basado en roles vincula facultades con funciones organizacionales. Ferraiolo y Kuhn (1992) desarrollaron RBAC como una alternativa apropiada para contextos organizacionales donde las responsabilidades de trabajo deben orientar el acceso.

El anteproyecto adopta explícitamente el control de acceso basado en roles. Esta elección guarda relación con la organización de las responsabilidades institucionales: una función administrativa puede persistir aunque cambie la persona que la desempeña. En lugar de reproducir una lista de permisos para cada cuenta, se define el conjunto de facultades de un rol y se asigna ese rol a las personas autorizadas. Así, el cambio de personal puede representarse modificando asignaciones, sin redefinir cada operación del sistema.

La comparación puede aplicarse a una función ilustrativa de administración de usuarios. Bajo un esquema individual, cada cuenta necesita una concesión independiente para crear, consultar y modificar registros. Bajo RBAC, esas capacidades pertenecen a una función administrativa y la pertenencia a ella produce el conjunto correspondiente. El proyecto utiliza `user_rol` y `rol_permissions` para representar las dos relaciones. La distinción tiene valor porque separa la identidad de la definición de sus responsabilidades digitales.

El control basado en atributos amplía las decisiones mediante características del sujeto, del recurso, de la operación y del entorno, según Hu et al. (2014). Su utilidad para SIPLAP Booked puede observarse cuando el departamento o el estado de una actividad debe restringir una acción. Un rol identifica una función general; un atributo puede determinar si esa función es aplicable al registro solicitado. Esta combinación requiere reglas expresas y no surge automáticamente por almacenar un departamento.

La elección de RBAC no elimina todos los problemas de autorización. Un rol demasiado amplio puede concentrar acciones innecesarias, y una proliferación de roles puede dificultar su revisión. Por ello, el modelo debe representar funciones comprensibles y mantenerse vinculado con las operaciones reales del sistema. La matriz de permisos constituye una herramienta para evaluar esa correspondencia.

En el proyecto, RBAC proporciona el fundamento de las relaciones de acceso. Las restricciones por cuenta activa y por contexto complementan ese fundamento. Su adecuación depende de que las asignaciones sean legítimas y de que el servidor utilice la política en las operaciones protegidas, con independencia de la forma en que se presente la interfaz.

6 Modelo formal de usuarios roles y permisos

RBAC puede explicarse mediante conjuntos y relaciones. Se denomina U al conjunto de usuarios, R al de roles y P al de permisos. La relación UA vincula usuarios con roles y la relación PA vincula roles con permisos. El modelo también contempla sesiones y activación de roles. NIST (s. f.) diferencia el núcleo de RBAC de sus extensiones jerárquicas y de restricciones, lo que permite analizar una implementación por los elementos que efectivamente incorpora.

Para el proyecto, users representa U, roles representa R y permissions representa P. Las tablas `user_roles` y `role_permissions` materializan UA y PA. Un permiso efectivo puede derivarse recorriendo ambas relaciones: se buscan los roles asignados a una cuenta y se reúnen los permisos de esos roles. Esta formulación explica por qué un permiso almacenado, pero no vinculado con una función de la cuenta, no produce autorización mediante la matriz.

Considérese un ejemplo de diseño con dos roles, OPERATIVO y GESTOR, y tres permisos, `activities:read`, `activities:create` y `activities:update`. Si OPERATIVO posee los dos primeros y GESTOR posee el primero y el tercero, una cuenta con ambos roles reúne las tres capacidades. La duplicación de `activities:read` no añade una facultad diferente; el resultado debe interpretarse como un conjunto sin repeticiones. Los nombres de este ejemplo ilustran una política posible y no implican que existan actualmente esas rutas.

Una expresión aplicable sería: permisos efectivos de u igual a la unión de los permisos de cada rol asignado a u. Esta operación describe la composición de concesiones. Para autorizar una solicitud todavía debe comprobarse que la cuenta esté activa y que se satisfagan las restricciones del recurso. También debe definirse si una operación exige cualquiera de varios permisos o todos ellos. La diferencia altera el significado de la política y merece una decisión explícita.

En el código examinado, RolesGuard utiliza `some` para evaluar alternativas dentro de la lista de roles y dentro de la lista de permisos. Si ambas categorías están presentes, las comprobaciones se ejecutan por separado. Sin embargo, los controladores administrativos emplean SuperUserGuard y el contexto construido por JwtAuthGuard recarga roles, sin incorporar una colección derivada de permisos. La estructura formal sirve así para distinguir el modelo persistido del mecanismo de aplicación visible.

La existencia de varias asignaciones tampoco prueba activación selectiva de roles ni herencia jerárquica. Para afirmar esas características se necesitarían relaciones y reglas específicas. Esta precisión teórica permite describir SIPLAP Booked como una base de usuarios, roles y permisos sin atribuirle automáticamente todas las propiedades de las extensiones de RBAC.

7 Mínimo privilegio y separación de funciones

El principio de mínimo privilegio establece que un sujeto debe disponer únicamente de las facultades necesarias para realizar sus tareas. La separación de funciones busca distribuir responsabilidades incompatibles para reducir conflictos de interés y concentraciones de poder (Saltzer y Schroeder, 1975; NIST, s. f.). Ambos conceptos permiten evaluar una matriz de acceso desde sus consecuencias organizacionales. En SIPLAP Booked, administrar cuentas, definir roles y asignar permisos son acciones sensibles porque modifican las capacidades de otros sujetos y pueden alterar indirectamente todo el sistema.

Una aplicación del mínimo privilegio consiste en diferenciar la consulta de usuarios de su creación o eliminación. Una persona que necesita localizar un responsable podría requerir información limitada, mientras que una función administrativa necesita modificar identidades. Reunir ambas necesidades bajo una sola autorización amplia simplifica la programación inicial, pero dificulta justificar las facultades de cada función. La granularidad debe responder a tareas identificadas y no solamente a la comodidad de reutilizar un control.

La separación de funciones puede formularse mediante una regla ilustrativa: quien propone una actividad no debe aprobar su propia propuesta cuando el proceso institucional exige revisión independiente. Para aplicarla, el servidor necesita conocer al creador, al aprobador y al registro. Asignar nombres distintos a los roles no es suficiente si una misma cuenta puede reunirlos y completar ambas acciones. La regla debe expresarse como una condición verificable sobre la operación.

En la administración de acceso, otro escenario sería someter las concesiones de facultades críticas a revisión por una segunda persona. Tal procedimiento representa una posibilidad de gobierno institucional y no una función confirmada del repositorio. Su valor teórico reside en diferenciar autorización para operar y autorización para cambiar la política. Una persona puede tener acceso a un módulo sin estar facultada para ampliar sus propias capacidades dentro de él.

El proyecto contiene una restricción concreta que impide a un superusuario retirarse su propio rol SUPERUSUARIO. Esta comprobación evita una forma de pérdida de acceso administrativo individual. No demuestra, por sí misma, una política completa de separación de funciones ni la protección de la última cuenta administradora en todos los escenarios. Su interpretación correcta es la de una restricción específica de negocio ubicada en el caso de uso correspondiente.

Estas nociones fundamentan revisiones periódicas de las asignaciones. Cuando una persona cambia de responsabilidad, sus facultades anteriores deben reevaluarse para evitar acumulación de privilegios. El objeto de la revisión no es únicamente confirmar que la cuenta exista, sino comprobar que cada capacidad conserve una justificación institucional y que las combinaciones de funciones no permitan completar operaciones incompatibles.

8 Gobierno de la matriz de permisos

Una matriz de permisos relaciona funciones con operaciones sobre recursos. Sus filas pueden representar roles y sus columnas acciones identificables. En el proyecto, el formato `recurso:accion` proporciona un vocabulario para expresar permisos, como `users:create`. El valor de la matriz consiste en convertir una decisión administrativa en una definición que pueda persistirse, revisarse y comprobarse. Su diseño exige explicar qué significa cada acción y cuál es su alcance, especialmente cuando una operación afecta a otras cuentas.

La construcción conceptual parte de las tareas institucionales. Primero se identifica la función que necesita una operación; después se define el recurso y la acción; finalmente se determina si existen restricciones adicionales. Por ejemplo, crear usuarios y asignar roles son operaciones relacionadas, pero diferentes. Conceder la primera no tendría que producir automáticamente la segunda. Esta distinción permite evitar privilegios implícitos y mejora la precisión de la revisión administrativa.

Los nombres de permisos funcionan como identificadores de política. Su estabilidad importa porque una modificación puede afectar las referencias utilizadas por los controles del servidor. En `packages/shared`, el esquema de slug exige una estructura normalizada con recurso y acción, mientras que la base de datos declara su unicidad. Estas dos medidas atienden problemas diferentes: el contrato controla la forma y la persistencia evita que dos registros utilicen el mismo identificador.

La asignación de un permiso a un rol debe interpretarse como una concesión con efectos sobre todos los usuarios vinculados con ese rol. Si diez cuentas poseen una función y se agrega una nueva facultad, las diez pueden quedar comprendidas por el cambio cuando el sistema derive la autorización de la matriz. Por ello, `modificar_role_permissions` tiene un alcance colectivo que debe considerarse en el gobierno de acceso. La pantalla de edición necesita hacer comprensible ese efecto.

Una matriz útil también documenta exclusiones y condiciones. En un ejemplo, un gestor podría editar actividades de su área mientras conserve un estado editable. La columna `activities:update` expresaría la capacidad general, pero no resolvería la propiedad del registro ni la transición de estado. Estas condiciones deben acompañar la definición del permiso o la especificación del caso de uso para impedir interpretaciones excesivamente amplias.

En SIPLAP Booked, la matriz cuenta con representación relacional y casos de uso para asignar permisos. La teoría exige un paso adicional: conectar esa representación con cada punto de autorización. Hasta que esa conexión sea verificable, la administración de la matriz constituye una capacidad de configuración y no evidencia suficiente de control granular en todos los servicios del sistema.

9 Gestión de usuarios y ciclo de vida de las identidades

Una identidad digital vincula una persona o un sujeto operativo con un identificador reconocido por el sistema. En SIPLAP Booked, users concentra el identificador único, correo, nombre de usuario, hash de contraseña y referencias organizacionales. La identidad debe permanecer distinguible durante su ciclo de vida: alta, cambios de información, asignación de responsabilidades, suspensión y retiro. Cada etapa puede modificar condiciones de acceso, pero no necesariamente requiere eliminar el registro que permite atribuir operaciones anteriores.

El alta de una cuenta establece una relación inicial entre identidad y credencial. El contrato compartido valida correo y contraseña, y el repositorio persiste el hash obtenido mediante el codificador. Conceptualmente, el correo cumple una función de localización para el ingreso, mientras que user_id proporciona una referencia estable para las relaciones. Cambiar el correo no debería crear una persona diferente si la operación conserva el identificador de la misma cuenta.

La asignación de funciones representa una etapa distinta. Una cuenta puede estar registrada y activa sin tener facultades administrativas. Esta situación evita confundir existencia con autorización y permite incorporar personal antes de definir todas sus responsabilidades. La relación user_roles conserva la asignación de cada función, con una clave compuesta que distingue los pares de usuario y rol. Así, el conjunto de responsabilidades puede variar sin reemplazar la identidad.

Los cambios organizacionales muestran la utilidad del ciclo de vida. Si una persona deja un departamento, debe revisarse su adscripción y las funciones que dependían de ella. Si abandona la institución, la desactivación puede impedir actuaciones futuras manteniendo las referencias existentes. La elección entre desactivar y eliminar requiere considerar las dependencias de datos; el esquema relaciona usuarios con actividades y reservaciones, por lo que una baja física puede encontrar restricciones de integridad.

En el repositorio de usuarios, la eliminación de asignaciones de roles y la eliminación de la cuenta se realizan dentro de una transacción. Esto evita completar solamente una parte de esa operación cuando falla la otra. Sin embargo, las referencias desde otras entidades deben respetarse según las reglas declaradas. La teoría del ciclo de vida ayuda a interpretar un rechazo por dependencia como protección de la consistencia, en lugar de asumir que toda eliminación debe forzarse.

La administración de identidades también requiere evitar cuentas compartidas cuando la atribución individual sea necesaria. Si varias personas utilizan una sola identidad, el sistema pierde precisión para explicar quién ejecutó una acción. El fundamento es organizacional: las capacidades digitales deben acompañar a sujetos identificables, y las modificaciones de acceso necesitan conservar coherencia con sus responsabilidades vigentes.

10 Perfiles departamentos tipos y estatus

El perfil, el departamento, el tipo y el estatus describen dimensiones diferentes de una cuenta. El perfil contiene información de presentación o contacto; el departamento expresa adscripción organizacional; el tipo permite clasificar personas según categorías institucionales; el estatus determina una condición operativa de la identidad. El anteproyecto incorpora estas dimensiones como módulos relacionados con el acceso (Euan Be, s. f., pp. 2-3). Distinguir sus significados evita que una propiedad descriptiva se convierta, sin una regla definida, en autorización.

En el esquema del proyecto, `profiles` comparte `user_id` como clave con `users`. Esta estructura representa un perfil como máximo por cuenta y permite que una cuenta no tenga todavía un registro de perfil. Sus campos incluyen nombres, apellidos, imagen, teléfono y otros datos personales. La relación uno a uno separa los atributos de presentación de las credenciales. Esta separación favorece respuestas diferenciadas para operaciones que necesitan identificar una cuenta y para pantallas que necesitan mostrar su información personal.

`Departments` agrupa al personal según áreas institucionales. Una adscripción puede ser útil para filtrar información, asignar responsables o establecer límites de gestión. No obstante, el departamento solo participa en la autorización si una regla lo utiliza explícitamente. Un usuario de un área no adquiere por esa referencia permiso para modificar todos sus registros. La condición organizacional debe combinarse con la función y con la relación respecto del objeto solicitado.

`User_types` permite representar una clasificación adicional, mientras que `roles` contiene funciones autorizadas. Aunque una institución podría relacionar ambas categorías mediante procedimientos de asignación, sus efectos no son idénticos. Un tipo describe cómo se clasifica una cuenta; un rol define un conjunto de capacidades. Mantener tablas separadas permite que una clasificación permanezca estable mientras cambian las funciones digitales de la persona.

`User_statuses` condiciona la elegibilidad de la cuenta. El ingreso y la comprobación de solicitudes protegidas verifican que el nombre del estatus sea `ACTIVE`. Esta comparación da un efecto operativo concreto a un catálogo. En consecuencia, los valores utilizados deben mantenerse consistentes: una etiqueta de presentación puede traducirse en la interfaz, pero la regla del servidor necesita reconocer la misma categoría de manera estable.

Los modelos de estas entidades están presentes en Prisma, pero su representación no acredita por sí sola servicios completos de gestión ni interfaces dedicadas. Para el marco teórico, su aportación es proporcionar los conceptos y relaciones necesarios para completar la base identitaria del sistema, preservando la diferencia entre información personal, pertenencia organizacional, clasificación y facultades de acceso.

11 Arquitectura web y comunicación entre cliente y servidor

La arquitectura web distribuye responsabilidades entre un cliente que presenta e inicia interacciones y un servidor que procesa solicitudes y aplica reglas. En SIPLAP Booked, frontend contiene la aplicación de Next.js y backend contiene la aplicación de NestJS. La comunicación utiliza solicitudes HTTP con cuerpos y respuestas estructurados. Esta separación permite que la presentación evolucione sin trasladar al navegador la autoridad para decidir sobre las operaciones institucionales.

HTTP define métodos, respuestas y códigos de estado que expresan la semántica de la comunicación (Fielding et al., 2022). GET se utiliza para obtener representaciones, mientras que POST, PUT, PATCH y DELETE permiten expresar diferentes operaciones sobre recursos. La respuesta informa si la petición se procesó o si encontró un problema. Para el proyecto, usar códigos coherentes ayuda a distinguir credenciales inválidas, falta de autorización, datos incorrectos y conflictos de integridad.

La frontera de confianza aparece cuando una solicitud abandona el navegador y llega al servidor. El cliente puede presentar opciones adecuadas al usuario, pero los datos recibidos necesitan validarse independientemente. Un formulario puede enviar un identificador manipulado o una petición puede construirse sin utilizar la interfaz oficial. Por ello, el servidor debe determinar la identidad a partir de evidencia verificable y aplicar las reglas antes de ejecutar el caso de uso.

Una petición de creación de usuario ilustra el recorrido. La interfaz reúne correo y contraseña; el cliente envía la solicitud; el servidor verifica la autenticación y el rol administrativo; el contrato valida la forma de los datos; el caso de uso prepara la operación; el repositorio solicita su persistencia. Una respuesta segura devuelve información de la cuenta sin exponer su contraseña ni su hash. Cada fase tiene un propósito diferente y su ubicación facilita la revisión.

El backend configura CORS con un origen previsto para el frontend. Esta configuración participa en la interacción del navegador con el servicio, pero no sustituye los controles de acceso. Un cliente ajeno al navegador puede formular solicitudes directamente. La autorización debe conservar su efecto incluso si la petición no está sujeta a las restricciones de lectura entre orígenes del navegador.

La arquitectura del proyecto no requiere describirse como un conjunto de microservicios por el hecho de separar frontend y backend. El servidor examinado organiza módulos dentro de una aplicación. Esta precisión evita atribuir una distribución operativa inexistente y mantiene el análisis centrado en las responsabilidades verificables: presentación, transporte, aplicación de reglas y almacenamiento de información.

12 Modularidad y arquitectura de puertos y adaptadores

La modularidad organiza el software en componentes con responsabilidades comprensibles y dependencias delimitadas. En el proyecto, los módulos de autenticación, usuarios, roles y permisos agrupan conceptos relacionados. Dentro de ellos aparecen directorios de dominio, aplicación, infraestructura y presentación. Esta estructura permite analizar dónde viven las reglas de negocio, cómo se ejecutan los casos de uso y qué componentes dependen de tecnologías concretas.

La arquitectura de puertos y adaptadores propuesta por Cockburn (2005) separa la lógica de aplicación de los mecanismos externos que la invocan o apoyan. Un puerto expresa una necesidad de interacción y un adaptador la implementa con una tecnología determinada. Su aplicación en SIPLAP Booked se observa en los contratos de repositorio, los servicios de token y las interfaces de comparación o codificación de contraseñas. Los casos de uso dependen de esas abstracciones y reciben implementaciones desde la infraestructura.

Por ejemplo, LoginUseCase necesita localizar una cuenta, comparar una contraseña y producir un token. No requiere conocer el detalle de una consulta Prisma ni la llamada específica a bcrypt. Esta separación permite razonar sobre la autenticación como una regla de aplicación: solo se entrega una credencial de sesión cuando la identidad existe, la evidencia es válida y la cuenta conserva el estado exigido. Los adaptadores realizan las operaciones técnicas necesarias para cumplir esa regla.

La organización por capas también delimita la traducción de errores. Un caso de uso puede expresar que un usuario no existe o que una asignación se encuentra duplicada. La presentación convierte ese resultado en una respuesta HTTP. Si la regla dependiera directamente de un código de transporte, sería más difícil utilizarla en otro contexto. Mantener errores de negocio diferenciados permite conservar su significado y adaptar la forma en que se comunican.

La inversión de dependencias no significa ausencia de dependencias, sino orientar la lógica central hacia contratos controlados por la aplicación. El repositorio implementa lo que el caso de uso necesita, y el módulo de NestJS conecta ambas partes. Así, una prueba puede sustituir el acceso a datos por un doble que reproduce resultados específicos. La sustitución sirve para examinar decisiones sin depender de todos los servicios externos.

La arquitectura debe evaluarse por sus efectos y no solo por los nombres de carpetas. En SIPLAP Booked, la presencia de entidades, puertos y adaptadores aporta evidencia de una separación deliberada. Su utilidad se refleja cuando una regla puede localizarse y probarse de manera independiente, mientras que los cambios de tecnología se concentran en los mecanismos que la aplicación utiliza para comunicarse con su entorno.

13 NestJS y aplicación central de las reglas de acceso

NestJS proporciona una organización del servidor mediante módulos, controladores y proveedores. Sus mecanismos de inyección de dependencias permiten relacionar contratos con implementaciones, y sus guards pueden evaluar condiciones antes de ejecutar un manejador de ruta (NestJS, s. f.-a, s. f.-b). En el proyecto, estos mecanismos conectan los casos de uso con repositorios Prisma y sitúan comprobaciones de identidad y rol en las operaciones administrativas.

Un controlador recibe la solicitud y representa el límite de transporte. Su función es obtener parámetros, validar entradas, invocar una operación de aplicación y devolver el resultado adecuado. El caso de uso expresa lo que la operación significa para el sistema. El repositorio encapsula el acceso a datos. Esta distribución permite comprender una modificación de usuarios sin atribuir al controlador toda la responsabilidad sobre autenticación, persistencia y reglas de negocio.

JwtAuthGuard actúa antes de la operación protegida. Extrae el token de la cabecera Authorization, verifica su validez, identifica al sujeto y consulta la cuenta actual. El resultado se incorpora al contexto de la solicitud. SuperUserGuard utiliza ese contexto para exigir SUPERUSUARIO. El orden conceptual es necesario: primero se establece una identidad válida y luego se examina si satisface la condición administrativa.

RolesGuard constituye otro mecanismo previsto en el código. Recupera metadatos de roles y permisos declarados para la ruta y evalúa alternativas mediante la información de la solicitud. Su existencia muestra una posibilidad de autorización más configurable. Sin embargo, si una ruta utiliza solamente SuperUserGuard, su decisión depende del rol administrativo y no de una consulta automática a todos los permisos registrados. La teoría de aplicación de políticas exige describir el mecanismo realmente conectado a cada operación.

También resulta relevante el tratamiento de rutas sin metadatos. RolesGuard permite continuar cuando no encuentra requisitos de roles o permisos. Este comportamiento puede ser apropiado para una ruta pública intencional, pero requiere inventariar las rutas que necesitan protección. La seguridad global no puede deducirse del comportamiento de un guard aislado; depende de cómo se aplican los controles a los recursos expuestos y de qué rutas se consideran públicas.

Los filtros de excepción y los pipes de validación complementan la frontera de presentación. Los primeros convierten errores en respuestas coherentes y los segundos comprueban datos antes de ejecutar acciones. Su relación con los guards muestra una defensa por etapas: verificar al solicitante, comprobar la autorización, validar el contenido y preservar la consistencia de la operación. Cada etapa responde a una pregunta diferente sobre la misma solicitud.

14 TypeScript contratos y validación de datos

TypeScript incorpora comprobaciones estáticas de tipos sobre programas de JavaScript, lo que ayuda a identificar incompatibilidades durante el desarrollo (Microsoft, s. f.). Los tipos describen estructuras esperadas para entidades, entradas y resultados. En SIPLAP Booked, frontend y backend utilizan TypeScript y comparten contratos mediante `packages/shared`. Esta organización permite expresar un vocabulario común para operaciones de usuarios, roles y permisos, reduciendo interpretaciones divergentes de sus campos.

La comprobación estática no verifica por sí misma una petición recibida durante la ejecución. Un objeto JSON enviado desde un cliente puede contener valores diferentes de los que el programa esperaba. Zod permite definir esquemas y validar datos en ese momento, además de inferir tipos a partir de ellos (Zod, s. f.). La combinación distingue dos necesidades: prevenir errores al escribir el programa y rechazar entradas inválidas al recibir solicitudes.

El contrato `createUserSchema` exige un correo válido y una contraseña con condiciones definidas. El nombre de usuario es opcional y tiene límites de longitud. Los esquemas de rol normalizan el nombre a mayúsculas, mientras que los de permiso normalizan el slug y comprueban su estructura. Estas transformaciones proporcionan una representación coherente para comparar identificadores y persistir información. Su significado debe conservarse en cliente y servidor.

Los objetos estrictos también restringen campos no previstos. En el alta de usuarios, el contrato público no permite incorporar libremente un arreglo de roles o un indicador administrativo. Esta decisión ayuda a impedir que el cliente introduzca capacidades a través de propiedades ajenas a la operación. La creación de la identidad y la asignación de funciones se tramitan mediante acciones distintas, lo que hace más visible el control de cada responsabilidad.

La validación sintáctica debe complementarse con reglas semánticas. Un UUID con formato correcto puede referirse a un usuario inexistente. Un nombre de rol normalizado puede duplicar otro ya registrado. Un permiso válido puede resultar incompatible con una política institucional. Por ello, los casos de uso y las restricciones de persistencia intervienen después de la validación del contrato. Ninguna etapa reemplaza las preguntas que corresponden a las otras.

Los esquemas de actualización admiten campos parciales y exigen que exista al menos un cambio. Esta regla evita peticiones vacías y representa la semántica de una modificación. Desde el fundamento teórico, un contrato compartido define acuerdos de interacción; la autorización decide quién puede utilizarlos; las reglas de negocio determinan si su contenido es admisible en el contexto del sistema. Mantener estas funciones separadas facilita explicar los rechazos y revisar la consistencia del diseño.

15 Modelo relacional y representación de la matriz de acceso

El modelo relacional representa información mediante relaciones compuestas por atributos y registros. Para analizar el proyecto, cada tabla corresponde con una entidad o con una asociación relevante. Users conserva identidades, roles conserva funciones y permissions conserva acciones identificadas. Las relaciones intermedias permiten expresar que una cuenta puede tener varias funciones y que una función puede contener varias facultades. Esta estructura evita limitar artificialmente el acceso a una sola categoría por persona.

La clave primaria identifica de manera inequívoca un registro. En users, user_id cumple esta función, mientras que email conserva una condición de unicidad independiente. Una clave foránea expresa una referencia hacia una entidad existente. PostgreSQL permite declarar estas restricciones y claves compuestas para mantener relaciones válidas (PostgreSQL Global Development Group, s. f.). Aplicadas al proyecto, impiden asignaciones que remitan a usuarios o roles inexistentes.

User_roles utiliza el par user_id y role_id como clave. La combinación permite múltiples roles para una cuenta y múltiples cuentas para un rol, pero evita repetir la misma asignación. Role_permissions utiliza de manera equivalente role_id y permission_id. Desde la interpretación de la matriz, cada fila de estas tablas constituye un vínculo explícito. El resultado de autorización puede obtenerse mediante recorridos entre relaciones, sin almacenar una copia independiente de todos los permisos por usuario.

La separación de catálogos favorece una representación coherente. Si el nombre de un rol se repitiera en cada registro de usuario, un cambio podría dejar valores divergentes. Mantener una entidad de rol y referencias hacia ella permite actualizar su descripción en un solo lugar. La relación de acceso permanece vinculada con su identificador, mientras que la etiqueta puede evolucionar conforme a necesidades de presentación o administración.

La cardinalidad debe corresponder con la regla de negocio. En profiles, compartir la clave user_id limita el perfil a uno por cuenta. En departments, varias cuentas pueden pertenecer a una misma área. En user_statuses, cada cuenta referencia una categoría de condición. Estas decisiones materializan supuestos distintos y permiten examinar qué configuraciones acepta el sistema antes de construir una interfaz de edición.

El modelo de datos no equivale a una política completa. Una clave foránea demuestra que una asignación apunta a entidades válidas, pero no que quien la creó tenía autoridad ni que la combinación de roles sea apropiada. La integridad relacional conserva coherencia estructural; los controles de aplicación conservan coherencia de actuación. SIPLAP Booked necesita ambas para que la matriz almacenada corresponda con decisiones legítimas y ejecutables.

16 Integridad transacciones y acceso a datos con Prisma

La persistencia debe conservar invariantes incluso cuando una operación requiere varios cambios. Una transacción agrupa acciones para que su resultado se confirme o se revierta como una unidad. La documentación de Prisma describe mecanismos transaccionales y alternativas para coordinar escrituras relacionadas (Prisma, s. f.). En SIPLAP Booked, este concepto es relevante para retirar asignaciones y eliminar entidades sin dejar modificaciones parciales que alteren la matriz de acceso.

Una eliminación de usuario ilustra el problema. Si el programa borra primero sus asignaciones y después falla al borrar la cuenta, un procedimiento sin atomicidad podría dejar al usuario existente, pero sin sus roles. La transacción permite que ambas acciones se interpreten como una operación conjunta. El repositorio del proyecto utiliza este mecanismo al eliminar las relaciones `user_rol` y la cuenta. La consistencia depende de conservar ese tratamiento cuando intervienen otras modificaciones vinculadas.

La unicidad también requiere protección frente a concurrencia. Dos solicitudes podrían consultar simultáneamente que un correo no existe y tratar de crearlo. La comprobación previa mejora el mensaje al usuario, pero la restricción de la base de datos resuelve el conflicto cuando ambas operaciones alcanzan la escritura. El adaptador debe traducir ese resultado en un error comprensible, manteniendo la causa de negocio sin exponer detalles internos del motor.

Prisma funciona como un mecanismo de acceso que relaciona el esquema con consultas tipadas. El proyecto declara PostgreSQL como proveedor y genera un cliente que sus repositorios utilizan. Esto permite concentrar la lógica de persistencia en adaptadores. El caso de uso puede solicitar una operación mediante su puerto y recibir un resultado de aplicación, mientras que el repositorio selecciona campos y convierte las estructuras de la base de datos.

La selección explícita de campos es significativa para la seguridad. Una consulta administrativa de usuarios puede recuperar identificadores, correo, estatus y roles sin incluir `password_hash`. El proyecto aplica selecciones de este tipo en sus repositorios. La decisión demuestra que almacenar un dato sensible y devolverlo al cliente son acciones independientes. El diseño de respuestas debe responder a lo que necesita la operación, no a todos los atributos disponibles en una tabla.

La existencia de modelos generados tampoco acredita servicios de negocio para cada entidad. Un archivo generado permite consultar una estructura, pero todavía se requieren contratos de entrada, reglas, controles y pruebas de su operación. Prisma aporta coherencia técnica entre esquema y acceso a datos; la aplicación define el significado institucional de cada cambio. Esta separación evita confundir capacidad de persistencia con funcionalidad completa del sistema.

17 Contraseñas y protección de credenciales

Una contraseña constituye evidencia de autenticación y necesita protección durante su almacenamiento. El hashing produce una representación que permite comprobar una candidata sin conservar el texto original para recuperarlo. OWASP recomienda algoritmos específicos para contraseñas y distingue ese tratamiento del cifrado reversible; para bcrypt advierte sobre su límite de entrada de 72 bytes y el factor de trabajo (OWASP, s. f.-c). Estos conceptos fundamentan decisiones concretas del proyecto.

SIPLAP Booked almacena password_hash y utiliza bcrypt. El adaptador de creación aplica un costo de diez, mientras que el adaptador de autenticación compara la contraseña recibida con el hash existente. La separación permite que el caso de uso trabaje con operaciones de codificación o comparación sin incluir detalles del algoritmo. El valor persistido es sensible y debe mantenerse fuera de las respuestas ordinarias y de los registros de diagnóstico.

El costo representa trabajo computacional para evaluar una contraseña. Su selección requiere considerar tanto la resistencia frente a intentos de adivinación como la capacidad del servidor para atender ingresos legítimos. El uso de bcrypt en el código describe una decisión vigente del proyecto; no implica que sea la elección preferida para todo sistema nuevo ni que su configuración haya sido evaluada bajo una carga institucional real. Una revisión de tecnología debe separar compatibilidad existente y criterios de actualización.

El contrato compartido comprueba la longitud mínima y el tamaño de la contraseña codificada en bytes. Esta distinción resulta relevante para texto con caracteres multibyte: el número de caracteres visibles puede ser menor que el tamaño de su representación. La regla busca que el valor aceptado sea compatible con el mecanismo utilizado. Debe conservarse de forma consistente en cliente y servidor, sin depender únicamente de un contador visual de caracteres.

La protección de credenciales comprende también su tratamiento durante errores. Una respuesta no necesita incluir la contraseña que falló ni el hash contra el que se comparó. Una herramienta de registro tampoco necesita almacenar el cuerpo completo de una solicitud de ingreso. Estas decisiones reducen la circulación de material sensible y mantienen la evidencia de acceso dentro de los componentes que deben procesarla.

En un ciclo de vida futuro, la recuperación de acceso requeriría un procedimiento diferente de mostrar la contraseña anterior. El fundamento del hash supone que la aplicación no dispone de ella en forma recuperable. Para el proyecto, ese principio orienta un eventual restablecimiento mediante evidencia temporal y verificación adecuada. Su descripción es una implicación de diseño, sin afirmar que dicho flujo esté implementado en el repositorio revisado.

18 Tokens JWT y validación de solicitudes

JSON Web Token es un formato para transmitir declaraciones sobre un sujeto u otro contexto. RFC 7519 define campos registrados como sub, iat y exp, que permiten identificar al sujeto y representar información temporal (Jones et al., 2015). Un JWT puede utilizar mecanismos de firma o cifrado según su construcción. En el proyecto se emplea como token de acceso verificado por el servidor, y su contenido no debe confundirse con una política de autorización permanente.

LoginUseCase solicita un token con el identificador del usuario, correo y roles. El servicio de infraestructura delega su generación a JwtService. Una solicitud posterior presenta ese valor en una cabecera Bearer y JwtAuthGuard lo verifica. Este recorrido evita enviar la contraseña en cada operación de negocio, pero convierte el token en una credencial que necesita protección durante su uso y transporte.

La validez criptográfica de un token y la vigencia administrativa de una cuenta son propiedades distintas. Un token pudo emitirse cuando la cuenta estaba activa y sus roles eran diferentes. Si el sistema utilizara únicamente las declaraciones originales, los cambios de acceso podrían tardar en surtir efecto. El guard del proyecto consulta la cuenta por sub y reemplaza el correo y los roles del contexto con información actual, además de verificar ACTIVE.

Este mecanismo permite razonar sobre revocación de facultades. Cuando una cuenta pierde SUPERUSUARIO, una petición administrativa posterior debe evaluarse con los roles actuales obtenidos del repositorio. El token mantiene la función de identificar una sesión, pero su lista histórica no constituye la autoridad exclusiva. Del mismo modo, la desaparición o desactivación de la cuenta impide que la verificación continúe como si la identidad conservara acceso vigente.

El tiempo de expiración representa otra dimensión. Un valor exp limita el período de aceptación, pero no sustituye la reevaluación de permisos. Tampoco basta decodificar el contenido del token: el servidor debe verificarlo con la configuración prevista. El proyecto utiliza verifyAsync y rechaza tokens ausentes, inválidos o expirados. La evaluación de todos los parámetros operativos exigiría revisar la configuración y el entorno concreto de despliegue.

La separación entre identificación por token y autorización actual proporciona una base para mantener consistencia durante cambios administrativos. Sin embargo, los permisos granulares todavía necesitan obtenerse o verificarse si una ruta depende de ellos. Recargar roles no equivale a calcular automáticamente toda la matriz. En SIPLAP Booked, la teoría de tokens explica la evidencia de sesión; la teoría de RBAC explica cómo esa identidad debe relacionarse con las facultades que se aplican a cada solicitud.

19 Sesiones y sincronización de la autorización

Una sesión mantiene continuidad entre interacciones de una identidad con la aplicación. Su gestión incluye creación, duración, validación y terminación. OWASP describe la relación entre autenticación, sesión y acceso, así como la necesidad de proteger las credenciales de sesión e invalidarlas conforme a su ciclo de vida (OWASP, s. f.-d). Para SIPLAP Booked, esta continuidad involucra tanto el cliente de Next.js como el servicio de autenticación del backend.

El frontend utiliza NextAuth con un proveedor de credenciales. Durante el ingreso consulta `/auth/login` y recibe información de la cuenta junto con el token del backend. Los callbacks trasladan estos datos al contexto de sesión que necesita la interfaz. Conceptualmente, se mantienen dos responsabilidades: el cliente conserva continuidad para presentar el estado de la persona y el servidor decide si cada operación protegida puede ejecutarse.

La sincronización importa cuando los datos de autorización cambian durante una sesión. En el código examinado, el callback `jwt` consulta `/auth/me` en interacciones posteriores para actualizar los roles y el correo. Si la consulta no confirma la sesión, el callback devuelve `null`. Esta conducta reduce la dependencia de los roles que la interfaz recibió originalmente y expresa una decisión restrictiva cuando no puede revalidarse la identidad.

Considérese una sesión de un administrador al que se retira su rol desde otra cuenta. La interfaz podría conservar temporalmente una vista administrativa abierta, pero el backend recarga los roles en la petición protegida y debe rechazar la acción. La actualización del cliente ayuda a reflejar ese cambio con claridad; la comprobación del servidor conserva la decisión efectiva. La consistencia visual y la consistencia de seguridad tienen objetivos relacionados, pero deben verificarse por separado.

También existe una diferencia entre cerrar la interfaz y retirar todas las credenciales aceptables. Si un token continúa válido bajo la política del servidor, ocultar una pantalla no lo revoca. Una estrategia completa debe definir qué cambios invalidan la sesión y cuáles se reflejan mediante consultas actualizadas. En el proyecto, la recarga de cuenta activa y roles aporta un mecanismo concreto; no demuestra por sí sola un catálogo completo de revocación de todos los tokens.

La configuración de secretos forma parte del entorno que sostiene las sesiones. Los valores reales deben proporcionarse mediante mecanismos de configuración controlados y preservarse fuera de la exposición pública. El fundamento teórico es que la confianza de una sesión depende de la capacidad del servidor para verificar su evidencia. El diseño de continuidad debe acompañar los cambios administrativos y comunicar de manera comprensible cuándo una identidad necesita volver a autenticarse.

20 React NextJS y separación de la presentación

React organiza interfaces mediante componentes y relaciones entre datos y estado, favoreciendo la descomposición de una pantalla en responsabilidades de presentación (React, s. f.). Next.js distingue componentes de servidor y cliente para organizar obtención de información e interactividad (Vercel, s. f.). En SIPLAP Booked, estas tecnologías permiten construir formularios, listas y controles de administración, mientras que las reglas decisivas de acceso permanecen en el backend.

La interfaz de administración incluye componentes para usuarios, roles, permisos y asignaciones. Esta separación refleja conceptos distintos del dominio y ayuda a que cada operación tenga una interacción reconocible. Un listado de roles puede mostrar funciones existentes; un componente de permisos por rol permite revisar las capacidades vinculadas con una función. La modularidad visual facilita reutilizar presentación sin mezclar el significado de las entidades.

El estado de la interfaz puede analizarse mediante fases de carga, resultado y error. Mientras se consulta información, la aplicación necesita indicar que la operación está en curso. Cuando termina, debe mostrar el resultado compatible con la autorización del usuario. Si falla, requiere conservar el contexto suficiente para comprender el problema. Estos estados permiten evitar que una ausencia temporal de datos se interprete como un catálogo vacío o como una acción completada.

Ocultar acciones según roles ayuda a orientar al usuario y reduce intentos que previsiblemente serán rechazados. Sin embargo, la condición visual es una representación de la política y puede quedar desactualizada. El servidor necesita comprobar la misma intención de acceso cuando llega la solicitud. En el proyecto, una vista administrativa visible no basta para crear un usuario si el backend no confirma la identidad activa y el rol requerido.

Las acciones que modifican autorización necesitan comunicar su alcance. Al asignar un permiso a un rol, la interfaz debería ayudar a comprender que el cambio afecta a las cuentas con esa función. Una operación sobre la matriz tiene consecuencias diferentes de editar una descripción. La teoría de presentación aplicada al proyecto permite justificar etiquetas claras, selección contextual y mensajes que expliquen el resultado de una concesión.

La separación entre cliente y servidor también ayuda a limitar información enviada al navegador. La interfaz solo necesita los campos necesarios para su tarea, y el servicio puede seleccionar una representación segura. Mostrar datos personales o detalles de infraestructura por conveniencia añade exposición sin mejorar necesariamente la interacción. En SIPLAP Booked, los componentes deben apoyar la administración del dominio y presentar el estado real de autorización, conservando un límite claro entre experiencia visual y autoridad operativa.

21 Usabilidad accesibilidad y comprensión del acceso

La usabilidad puede analizarse en este proyecto como la facilidad con que una persona comprende y completa tareas de administración bajo condiciones definidas. Crear una cuenta o asignar una función requiere que la interfaz presente información suficiente para anticipar el resultado. Una operación técnicamente correcta puede producir errores humanos si sus etiquetas son ambiguas o si no comunica qué cuenta, rol o permiso está modificando.

La accesibilidad incorpora condiciones para que distintas personas puedan percibir y operar el contenido. WCAG 2.2 organiza sus pautas alrededor de contenido perceptible, operable, comprensible y robusto (World Wide Web Consortium, 2024). Su aplicación al proyecto orienta criterios como identificar campos, conservar navegación por teclado, mostrar foco visible y comunicar errores mediante información que no dependa exclusivamente del color. Estos criterios son referencias de diseño, sin acreditar una evaluación completa de conformidad del sistema.

En un formulario de ingreso, una etiqueta permanente facilita reconocer el dato solicitado. Un mensaje asociado con el correo permite corregir su formato, mientras que un mensaje general puede explicar que la autenticación no se completó. La interfaz debe distinguir una dificultad de conectividad de una condición administrativa cuando el servidor proporcione esa categoría. Esta separación evita que el usuario repita acciones que no resolverán la causa del rechazo.

Para asignar roles, la comprensión exige mostrar el destinatario y la función seleccionada. Dos cuentas con nombres parecidos necesitan atributos suficientes para identificarse sin exponer información innecesaria. El correo o un identificador institucional autorizado pueden ayudar a distinguirlas. La decisión visual debe responder al contexto de la tarea y a la sensibilidad de los datos, en lugar de añadir campos personales por defecto.

Las acciones destructivas o de alto impacto necesitan una interacción que haga visible su consecuencia. Eliminar una cuenta difiere de retirar un rol; lo primero afecta a la identidad y sus referencias, mientras que lo segundo cambia una capacidad. Un texto genérico de éxito puede ocultar esa diferencia. Comunicar el resultado con el nombre de la operación ayuda a que la persona revise si el sistema ejecutó lo que pretendía.

La experiencia del usuario también debe acompañar cambios de autorización. Si una sesión pierde facultades, la aplicación debe actualizar sus opciones y explicar el rechazo de una acción pendiente. Mantener controles visibles que ya no pueden utilizarse produce confusión, aunque el backend conserve la seguridad. La relación entre accesibilidad, usabilidad y control de acceso permite fundamentar interfaces que guíen al personal institucional y reduzcan errores de administración de la matriz.

22 Requisitos y calidad del software

Los requisitos describen comportamientos esperados y condiciones que el producto debe cumplir. Un requisito funcional puede establecer que una cuenta administradora asigne un rol; un requisito de calidad puede exigir que el sistema rechace asignaciones inválidas y conserve consistencia ante fallos. ISO/IEC 25010:2023 proporciona un modelo de calidad del producto que sirve como referencia para especificar y evaluar propiedades del software (ISO e IEC, 2023). Su uso conceptual no implica certificación del proyecto.

El anteproyecto organiza actividades de análisis, diseño, programación, pruebas, corrección y documentación. Esta secuencia permite relacionar cada necesidad con una regla implementable y una comprobación. Por ejemplo, la necesidad de restringir administración se transforma en una condición sobre el rol, se conecta con un guard y se verifica mediante una petición de una cuenta sin ese rol. La trazabilidad mantiene la relación entre lo solicitado y la evidencia que permite evaluarlo.

Los requisitos necesitan condiciones observables. Expresar que el sistema debe ser seguro ofrece una intención, pero no define por sí mismo una prueba. Una formulación más precisa sería que las rutas administrativas rechacen solicitudes sin autenticación válida y solicitudes de cuentas que no tengan SUPERUSUARIO. De forma equivalente, la integridad de la matriz puede expresarse exigiendo que una asignación duplicada no produzca un segundo vínculo y que una referencia inexistente no se persista.

La calidad también incluye capacidad de mantenimiento. Si una regla de asignación se encuentra en un caso de uso y depende de un contrato definido, su revisión puede concentrarse en una unidad comprensible. Si la misma regla se copia en pantallas y controladores sin un criterio común, los cambios pueden producir divergencias. La arquitectura del proyecto aporta una base para localizar comportamientos y examinar su relación con los requisitos.

El desarrollo seguro requiere incorporar prácticas durante el ciclo de vida y no únicamente al final. El marco SSDF de NIST presenta recomendaciones para reducir riesgos de vulnerabilidades mediante preparación, protección del software, producción de software seguro y respuesta a problemas identificados (Souppaya et al., 2022). En SIPLAP Booked, esta orientación permite justificar revisión de contratos, controles del servidor, pruebas de autorización y gestión de configuraciones sensibles como actividades conectadas con el desarrollo.

La evaluación debe separar objetivos y resultados. La existencia de una prueba definida permite describir qué se pretende verificar; su ejecución documentada permite afirmar qué ocurrió bajo unas condiciones. Para un reporte académico, esta distinción conserva precisión: el marco teórico fundamenta los criterios y la sección de resultados deberá presentar evidencias de cumplimiento, fallos encontrados y correcciones realmente efectuadas.

23 Pruebas de autenticación autorización e integridad

Las pruebas permiten contrastar el comportamiento del software con condiciones esperadas. En un sistema de acceso, los casos autorizados muestran que una función legítima puede ejecutarse, mientras que los casos de rechazo muestran que las restricciones tienen efecto. Vitest ofrece mecanismos para organizar y ejecutar pruebas de programas JavaScript y TypeScript (Vitest, s. f.). El proyecto lo utiliza para pruebas del backend y también incluye suites HTTP y un escenario específico con PostgreSQL.

Una prueba unitaria puede examinar LoginUseCase con repositorios y servicios sustituidos. Los escenarios incluyen cuenta inexistente, contraseña incorrecta, estatus inactivo y autenticación válida. La sustitución permite controlar cada condición y observar si el caso de uso produce el resultado esperado. Su alcance es la decisión de aplicación; no verifica por sí sola la configuración completa de HTTP ni el comportamiento de la base de datos real.

Las pruebas HTTP atraviesan más componentes. En `backend/test/rbac.http.test.mjs` se utilizan módulos, guards, validadores, casos de uso y adaptadores, mientras que la frontera de datos se sustituye por un doble en memoria. Esta configuración permite revisar respuestas y encadenamiento de controles sin modificar registros reales. La separación debe conservarse al interpretar resultados: la suite prueba integración de la aplicación con un mecanismo de persistencia simulado, no todos los comportamientos de PostgreSQL.

La matriz de pruebas debe derivarse de la matriz de acceso. Para una ruta administrativa, pueden compararse solicitudes sin token, con token inválido, con identidad activa sin rol requerido y con superusuario activo. En las modificaciones de roles, también conviene considerar cambios durante una sesión, asignaciones duplicadas y referencias inexistentes. Cada escenario necesita un resultado esperado que pueda atribuirse a una regla concreta.

La integridad relacional requiere pruebas con el motor real cuando se pretende evaluar restricciones y transacciones propias de la base de datos. El archivo `rbac.postgres.mjs` proporciona un punto de trabajo para ese nivel. Su presencia indica una estrategia de verificación disponible, pero no acredita que todas sus pruebas hayan sido ejecutadas satisfactoriamente. El marco teórico distingue herramientas existentes y evidencia experimental.

La cobertura de código ayuda a localizar partes que no se ejercitan, aunque no equivale a cobertura de políticas. Una misma rama puede evaluarse con un único usuario sin examinar todas las combinaciones de roles. Para SIPLAP Booked, la prueba significativa es la que conecta sujeto, operación y condición con una respuesta verificable. La calidad del conjunto depende de representar las decisiones que podrían permitir una actuación indebida o bloquear una función legítima.

24 Trazabilidad auditoría y protección de información personal

La trazabilidad permite reconstruir relaciones entre decisiones, sujetos y operaciones. En el ámbito del acceso, interesa saber quién modificó una cuenta, qué función asignó y cuándo ocurrió. La auditoría utiliza información registrada para revisar esos hechos bajo criterios definidos. OWASP recomienda que los registros de aplicación contemplen eventos relevantes de seguridad y eviten incluir credenciales o datos sensibles innecesarios (OWASP, s. f.-b). La finalidad es disponer de evidencia útil sin ampliar la exposición de información.

Un evento de asignación de rol podría incluir el identificador del actor, el de la cuenta destinataria, el rol, la fecha y el resultado. La descripción permite diferenciar una acción completada de un intento rechazado. Si solo se conserva que una fila se actualizó, no siempre puede establecerse quién inició el cambio ni cuál era su intención. Por ello, el diseño de auditoría debe relacionarse con operaciones de aplicación y no únicamente con modificaciones técnicas.

En el esquema examinado aparecen `created_at` y `updated_at` en varias entidades. Estos campos proporcionan información temporal, pero no constituyen una bitácora completa de cambios. Un valor `updated_at` puede señalar la última actualización sin conservar las versiones anteriores, el actor o el motivo. La distinción resulta importante para el objetivo de trazabilidad del anteproyecto: las referencias temporales aportan una base, pero la reconstrucción de decisiones requiere información adicional.

La información personal también necesita delimitarse por finalidad. Profiles contiene atributos que pueden apoyar identificación, contacto o presentación. Una consulta de administración de roles quizá no necesite fecha de nacimiento ni teléfono. Diseñar respuestas según la tarea permite reducir datos transferidos y evita que una capacidad sobre autorizaciones se interprete como acceso irrestricto al perfil personal. Este principio se formula aquí como criterio técnico de diseño, sin afirmar una evaluación jurídica de cumplimiento.

La conservación de eventos debe corresponder con su utilidad y con reglas institucionales validadas. Retener todo de manera indefinida puede generar exposición innecesaria; conservar demasiado poco dificulta investigar un cambio indebido. El plazo, los responsables de consulta y la protección de los registros requieren decisiones explícitas. Los eventos de auditoría también necesitan autorización, ya que pueden revelar información sobre cuentas y actividades.

En SIPLAP Booked, la trazabilidad debe conectar identidad autenticada, acción autorizada y persistencia del resultado. Una política de acceso bien definida facilita interpretar cada evento, porque proporciona el criterio esperado para la actuación. La auditoría completa es una línea de desarrollo que debe demostrarse mediante servicios y registros verificables; no puede deducirse únicamente de marcas temporales o del hecho de utilizar JWT.

25 Integración de los fundamentos en SIPLAP Booked

Los fundamentos desarrollados permiten explicar el control de acceso como una relación entre identidad, función, operación y contexto. La cuenta identifica a quien actúa; el rol representa facultades asociadas con una función; el permiso describe una acción; las condiciones organizacionales delimitan cuándo y sobre qué registros puede aplicarse. Esta relación vincula el propósito del anteproyecto con la arquitectura y el modelo de datos de SIPLAP Booked.

La base conceptual de usuarios se materializa en users y en el procedimiento de autenticación. La diferenciación entre información personal y autorización se refleja en profiles, departments, user_types y user_statuses. El fundamento de RBAC aparece en roles, permissions, user_roles y role_permissions. Estas correspondencias permiten justificar las entidades del proyecto a partir de su significado, en lugar de presentar el esquema como una lista de tablas sin relación con necesidades institucionales.

La arquitectura por módulos y capas conecta esos conceptos con decisiones de implementación. Los contratos compartidos describen entradas válidas; los casos de uso representan operaciones; los guards examinan condiciones de acceso; los repositorios persisten resultados. La selección de campos y las restricciones de base de datos complementan los controles de aplicación. La consistencia depende de que cada componente conserve su responsabilidad y de que sus interacciones representen la misma política.

En la versión examinada, la administración de usuarios, roles y permisos se protege principalmente mediante la cuenta activa y el rol SUPERUSUARIO. Existe una matriz relacional y un guard para requisitos más granulares, pero derivar permisos efectivos y aplicarlos en las rutas requiere conexiones verificables. Esta observación delimita el alcance actual sin desestimar la utilidad del modelo: la estructura proporciona un fundamento para ampliar la autorización, siempre que la política se incorpore al recorrido real de las solicitudes.

Los fundamentos también permiten definir criterios de evaluación. Una identidad inactiva no debe actuar en rutas protegidas; un cambio de roles debe reflejarse en la autorización posterior; una asignación inválida no debe persistirse; la interfaz debe comunicar resultados comprensibles; los datos sensibles deben quedar fuera de respuestas innecesarias. La aplicación de estas condiciones necesita pruebas documentadas y revisión del entorno operativo para sostener afirmaciones de cumplimiento.

El marco teórico sitúa así la aportación de la residencia en la construcción de una base de acceso para la planeación institucional. La continuidad del proyecto depende de traducir responsabilidades de la Universidad en políticas explícitas, mantener una representación relacional coherente y comprobar cada operación del servidor. Estos fundamentos sirven para orientar el desarrollo de los módulos previstos y para relacionar sus resultados con seguridad, trazabilidad, capacidad de mantenimiento y uso institucional.

Referencias

- Cockburn, A. (2005). Hexagonal architecture. <https://alistair.cockburn.us/hexagonal-architecture>
- Euan Be, J. A. (s. f.). Módulos de Control de Acceso Basado en Roles del Sistema de Planeación Institucional [Anteproyecto de residencia profesional, Instituto Tecnológico Superior de Valladolid]. Documento proporcionado por el autor.
- Ferraiolo, D. F., y Kuhn, D. R. (1992). Role-Based Access Controls. 15th National Computer Security Conference. National Institute of Standards and Technology. <https://www.nist.gov/publications/role-based-access-controls>
- Fielding, R., Nottingham, M., y Reschke, J. (2022). HTTP Semantics (RFC 9110). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9110>
- Hu, V. C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., y Scarfone, K. (2014). Guide to Attribute Based Access Control (ABAC) Definition and Considerations (NIST SP 800-162; actualización de 2019). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-162>
- International Organization for Standardization e International Electrotechnical Commission [ISO e IEC]. (2023). ISO/IEC 25010:2023 Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model [Ficha y resumen público]. <https://www.iso.org/standard/78176.html>
- Jones, M., Bradley, J., y Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>
- Microsoft. (s. f.). The TypeScript Handbook. <https://www.typescriptlang.org/docs/handbook/intro.html>
- National Institute of Standards and Technology [NIST]. (s. f.). Role Based Access Control FAQs. <https://csrc.nist.gov/projects/role-based-access-control/faqs>
- NestJS. (s. f.-a). Authorization. <https://docs.nestjs.com/security/authorization>
- NestJS. (s. f.-b). Custom providers. <https://docs.nestjs.com/fundamentals/custom-providers>
- OWASP. (s. f.-a). Authorization Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- OWASP. (s. f.-b). Logging Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- OWASP. (s. f.-c). Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP. (s. f.-d). Session Management Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- PostgreSQL Global Development Group. (s. f.). Constraints. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/ddl-constraints.html>

Prisma. (s. f.). Transactions. <https://www.prisma.io/docs/orm/fundamentals/transactions>

React. (s. f.). Thinking in React. <https://react.dev/learn/thinking-in-react>

Saltzer, J. H., y Schroeder, M. D. (1975). The protection of information in computer systems. Proceedings of the IEEE, 63(9), 1278-1308. <https://www.cs.virginia.edu/~evans/cs551/saltzer/>

SIPLAP Booked. (2026). Código fuente del proyecto siplap-booked [Repositorio local, consulta del 2 de octubre de 2026]. Materiales examinados: backend/prisma/schema.prisma; backend/src/auth; backend/src/users; backend/src/roles; backend/src/permissions; packages/shared/src/index.ts; frontend/src/lib/auth.ts; frontend/src/components; backend/test; manifiestos package.json. Las descripciones de implementación de este capítulo corresponden a dichos materiales.

Souppaya, M., Scarfone, K., y Dodson, D. (2022). Secure Software Development Framework (SSDF) Version 1.1 Recommendations for Mitigating the Risk of Software Vulnerabilities (NIST SP 800-218). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-218>

Vercel. (s. f.). Server and Client Components. Next.js Documentation. <https://nextjs.org/docs/app/getting-started/server-and-client-components>

Vitest. (s. f.). Getting Started. <https://vitest.dev/guide/>

World Wide Web Consortium. (2024). Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation, 12 de diciembre de 2024. <https://www.w3.org/TR/WCAG22/>

Zod. (s. f.). Basic usage. <https://zod.dev/basics>

Las páginas web sin fecha se consultaron el 2 de octubre de 2026.